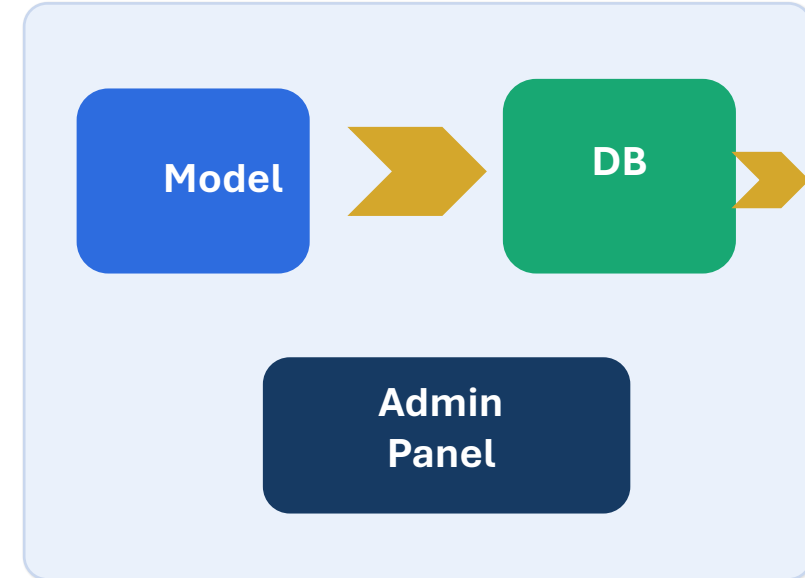


## Lecture 8

### Models, Database, Migrations, and Admin

*Django turns Python classes into database-powered web applications.*



## Learning Objectives

*What students should be able to do by the end of the lecture*

Goals

**1. Define what a Django model is and why it matters**

**2. Map Python classes to database tables**

**3. Create fields and choose correct data types**

**4. Understand migrations and their workflow**

**5. Register models in the admin panel**

**6. Create, read, update, and delete data conceptually**

## Quick Recap

*How today connects to previous lectures*

### Lecture 5

- Created a Django project
- 
- Ran development server
- 
- Understood settings and structure

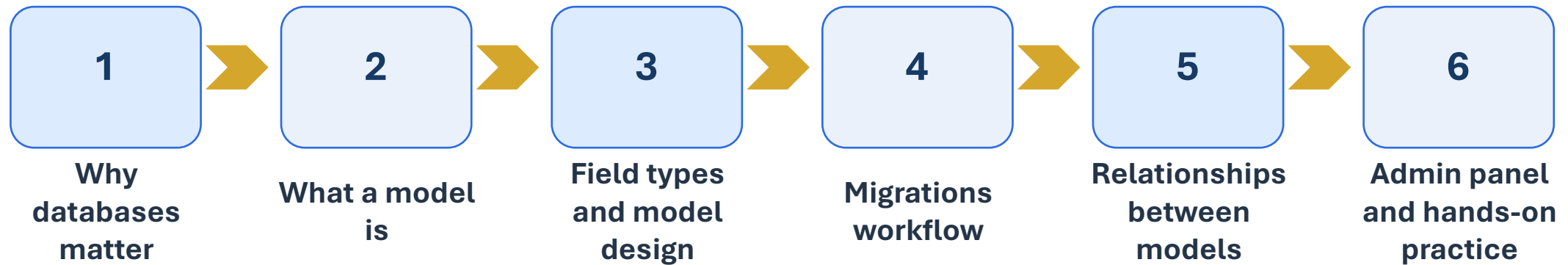
### Lecture 6

- Created apps
- 
- Connected URLs
- 
- Wrote views and responses

### Lecture 7

- Rendered templates
- 
- Passed context
- 
- Built dynamic HTML pages

## Today's Roadmap



## Why Do We Need a Database?

*Persistence means data survives after the program stops*

- Variables live only while the program is running
- A database keeps data for tomorrow, next week, and next semester
- Multiple users can access the same stored information
- Searching, filtering, and updating become easier

### Without Database

```
student_list = ["Ali", "Mona"]  
# Data disappears after  
restart
```

### With Database

Students table keeps records permanently  
Name | Level | Department

## Memory vs Database

*Temporary storage is not enough for web applications*

### Program Memory

- Fast and temporary
- Lost when server restarts
- Good for current processing
- Not shared as permanent records

### Database Storage

- Persistent and structured
- Survives restart
- Shared by users and system parts
- Supports queries, relations, and history

## What Is a Django Model?

*A model is a Python class that describes stored data*

- Written as a Python class
- Usually inherits from `models.Model`
- Each attribute becomes a database field
- Each object becomes one row in the table

```
1 from django.db import models
2
3 class Student(models.Model):
4     name = models.CharField(max_length=100)
5     level = models.IntegerField()
```

**Python class → database table**

## Model = Blueprint for Data

**Class  
Student**



### Student Table

|            |     |
|------------|-----|
| id         | 1   |
| name       | Ali |
| level      | 2   |
| department | CS  |



**One Row =  
One  
Object**

## First Model Example

*A simple Student model*

- Import models from django.db
- Inherit from models.Model
- Define fields with types and options
- Add `__str__` for readable display

```
1 from django.db import models
2
3 class Student(models.Model):
4     name = models.CharField(max_length=100)
5     age = models.IntegerField()
6     department = models.CharField(max_length=50)
7
8     def __str__(self):
9         return self.name
```

## Class-to-Table Mapping

*Translate each programming concept into database language*

### Django Code

```
class Student(models.Model)
```



### Database Meaning

**Table**

```
name = models.CharField(...)
```



**Column**

```
Student(name="Ali")
```



**Object / Row data**

```
__str__ method
```



**Readable display only**

## Default Database in Django

*SQLite is ready to use for learning and small projects*

- No extra installation in most setups
- Stored as a local file (db.sqlite3)
- Perfect for classroom demos and prototypes
- Easy to start, but not always ideal for large production systems

### settings.py

```
1 DATABASES = {  
2     'default': {  
3         'ENGINE':  
4         'django.db.backends.sqlite3',  
5         'NAME': BASE_DIR / 'db.sqlite3',  
6     }  
}
```

Result: a local database file inside the project folder.

## Other Databases You May Hear About

**SQLite**

Excellent for learning and small local projects

**PostgreSQL**

Popular choice for serious Django production projects

**MySQL**

Widely used in many web systems

**Oracle /  
Others**

Less common in beginner courses but possible

**For this course: SQLite is enough.**

## Where Do Models Live?

*Models are usually written inside models.py of an app*

### Project

#### Structure

```
myproject/  
├── manage.py  
├── myproject/  
└── students/  
    ├── admin.py  
    ├── apps.py  
    ├── models.py ← here  
    ├── views.py  
    └── migrations/
```

- Each Django app can own its models
- Keep related data together inside the same app
- Example: students app contains Student model
- Later, admin.py often imports models from models.py

## Writing a Model Step by Step

*From import to field definitions*

- Open the app folder
- Go to models.py
- Import models
- Write class Student(models.Model)
- Add fields one by one
- Save the file before migrations

```
1 # students/models.py
2 from django.db import models
3
4 class Student(models.Model):
5     name = models.CharField(max_length=100)
6     level = models.IntegerField()
7     department = models.CharField(max_length=50)
```

## Why Inherit from `models.Model`?

- It tells Django this class should be managed as a database model
- Django adds many features automatically
- The class gets ORM abilities such as `save()`, `delete()`, objects
- Without it, the class is just a normal Python class

```
1 # Normal Python class
2 class Student:
3     pass
4
5 # Django model
6 class Student(models.Model):
7     pass
```

**Inheritance gives the class database behavior.**

## Common Field Types in Django

### **CharField**

Short text such as name or department

### **TextField**

Long text such as description or notes

### **IntegerField**

Whole numbers such as level or age

### **BooleanField**

True / False values

### **DateField / DateTimeField**

Dates and timestamps

### **EmailField**

Email input with suitable meaning

## CharField

*Use it for short text with max\_length*

- Good for names, codes, titles, departments
- 
- Requires max\_length
- 
- Stored as short text in the database
- 
- Often one of the most common fields

```
1 name = models.CharField(max_length=100)
2 code = models.CharField(max_length=10)
3 department = models.CharField(max_length=50)
```

## TextField, IntegerField, and DateTimeField

- TextField for large text
- 
- Example: description, notes
- 
- Usually no max\_length requirement

- IntegerField for whole numbers
- 
- Example: age, level, credit hours
- 
- Use numeric meaning, not text

- DateTimeField for date + time
- 
- Example: created\_at, updated\_at
- 
- Useful for tracking records

## Primary Key and id Field

- Django automatically adds an id field unless you define your own primary key
- The id uniquely identifies each row
- Primary keys prevent ambiguity between records
- 
- Usually you will see values like 1, 2, 3, 4...

### Student Table

| id | name | level |
|----|------|-------|
| 1  | Ali  | 2     |
| 2  | Mona | 2     |
| 3  | Omar | 1     |

## Why `__str__` Matters

*Readable display in admin and shell*

- Returns a human-friendly text representation
- Improves object display in admin panel
- Does not create a database column
- Often returns name or title

```
1 class Student(models.Model):
2     name = models.CharField(max_length=100)
3
4     def __str__(self):
5         return self.name
```

## Important Field Options

*null, blank, and default*

### **null=True**

- Database can store NULL
- Mostly about database level

### **blank=True**

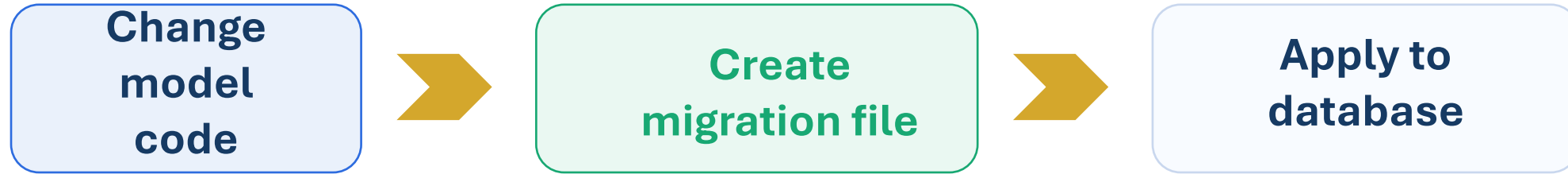
- Field can be empty in forms
- Mostly about validation level

### **default=**

- Provides automatic initial value
- Useful when field should not be empty

## What Are Migrations?

*Migrations are versioned instructions for database changes*



*Think of migrations as a history of database structure changes.*

## Migration Workflow



## makemigrations

*Creates migration files based on model changes*

- Compares current models with previous migration state
- Generates Python migration files inside migrations folder
- Does not change the database yet
- Useful to inspect what Django understood

```
1 python manage.py makemigrations
```

## migrate

*Applies pending migrations to the database*

- Reads migration files in order
- Creates or alters database tables
- Updates Django's migration history
- Must be run whenever structure changes

```
1 python manage.py migrate
```

## Helpful Migration Commands

```
python manage.py
```

```
showmigrations
```

Shows which migrations are applied

```
python manage.py sqlmigrate  
students 0001
```

Shows the SQL generated for a migration

- `showmigrations` helps debug which steps have run
- `sqlmigrate` helps advanced learners see the SQL behind Django ORM

## Common Migration Problems

- Changed model but forgot migrate
- **Added non-null field to existing table without default**
- Edited old migration files carelessly
- Database and migration history got out of sync

**Rule of thumb: change models carefully, then create and apply migrations immediately.**

## Modifying an Existing Model

*Add a new field, then migrate again*

- Edit the model code
- Run makemigrations
- Run migrate
- Repeat this pattern whenever the structure changes

```
1 class Student(models.Model):  
2     name = models.CharField(max_length=100)  
3     level = models.IntegerField()  
4     email = models.EmailField(blank=True)
```

## Design Decisions: Required or Optional?

- Required fields make sense for essential information
- Optional fields fit less critical information
- Defaults reduce repetitive input
- Think about the real business rule before coding

```
1 department =  
models.CharField(max_length=50)  
2 email = models.EmailField(blank=True)  
3 level = models.IntegerField(default=1)
```

**Ask: which fields are truly mandatory in a student record?**

## Relationships Between Models

*Real systems have connected data, not isolated tables*

### Student ↔ Course ↔ Instructor ↔ Department

- A student belongs to a department
- A course may have many students
- A profile may belong to exactly one student
- Relationships are critical in backend design

## ForeignKey

*Many objects point to one related object*

- Common for many-to-one relationships
- Example: many students belong to one department
- Stored as a reference to another table row
- Very common in Django apps

```
1 class Department(models.Model):
2     name = models.CharField(max_length=100)
3
4 class Student(models.Model):
5     name = models.CharField(max_length=100)
6     department = models.ForeignKey(Department,
on_delete=models.CASCADE)
```

## OneToOneField

*Exactly one related object for exactly one main object*

- Good for profile details or extension tables
- Example: one student has one ID card record
- Separates optional or extra data from main table
- Each side maps to one record

```
1 class Student(models.Model):
2     name = models.CharField(max_length=100)
3
4 class StudentProfile(models.Model):
5     student = models.OneToOneField(Student,
6                                     on_delete=models.CASCADE)
7     address = models.CharField(max_length=200)
8     phone = models.CharField(max_length=20)
```

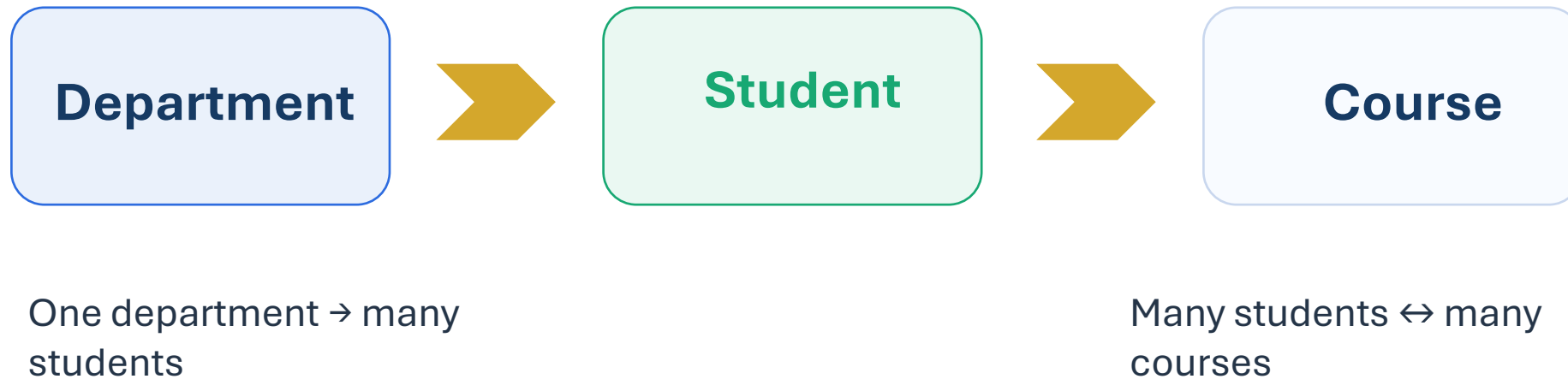
## ManyToManyField

*Many records on both sides of the relationship*

- A student can take many courses
- A course can include many students
- Django creates an intermediate table automatically
- Excellent for registration-like systems

```
1 class Course(models.Model):  
2     title = models.CharField(max_length=100)  
3  
4 class Student(models.Model):  
5     name = models.CharField(max_length=100)  
6     courses = models.ManyToManyField(Course)
```

## Relationship Diagram Example



## Django ORM: Talking to the Database with Python

- ORM = Object Relational Mapper
- 
- Lets us work with Python objects instead of writing raw SQL all the time
- 
- Creates, reads, updates, and deletes data
- 
- Understands model relationships

```
1 Student.objects.create(name='Ali',  
level=2)  
2 Student.objects.filter(level=2)  
3 Student.objects.get(id=1)
```

**Python code → SQL behind the scenes**

## Create Data with ORM

*Insert new rows into the database*

- Use `.create(...)` for quick creation
- 
- Or create object then `save()`
- 
- Field names must match model fields
- 
- Creates a new row in the related table

```
1 Student.objects.create(name='Ali', level=2,  
department='CS')  
2  
3 # another style  
4 student = Student(name='Mona', level=1,  
department='AI')  
5 student.save()
```

## Read Data: `all()` and `filter()`

*Retrieve records from the database*

- `all()` returns all records
- 
- `filter()` returns matching records only
- 
- Results can be displayed in views and templates
- 
- Filtering is essential in real applications

```
1 Student.objects.all()
2 Student.objects.filter(level=2)
3 Student.objects.filter(department='CS')
```

## Read One Record with get()

*Use get() when you expect exactly one object*

- Often used with id or unique fields
- 
- Raises an error if no record or multiple records match
- 
- Suitable when fetching a single detail page

```
1 student = Student.objects.get(id=1)
2 print(student.name)
```

## Update Data

*Modify existing records then save again*

- Fetch the object first
- 
- Change one or more fields
- 
- Call `save()` to persist changes
- 
- Very common in forms and admin editing

```
1 student = Student.objects.get(id=1)
2 student.level = 3
3 student.department = 'IS'
4 student.save()
```

## Delete Data

*Remove a record from the database*

- Fetch the object first
- 
- Call delete()
- 
- Use carefully because data may be lost
- 
- Admin panel also supports deletion

```
1 student = Student.objects.get(id=3)
2 student.delete()
```

## What Is Django Admin?

*A built-in management interface for your models*

- Comes with Django out of the box
- 
- Lets admins add, edit, and delete records
- 
- Excellent for testing and internal management tools
- 
- Saves time during development

**Imagine a ready-made  
dashboard  
for your models and  
tables.**

## Register a Model in admin.py

*Without registration, the model will not appear*

- Import the model in admin.py
- 
- Use `admin.site.register(ModelName)`
- 
- Then open the admin panel after migrations
- 
- Simple but essential step

```
1 from django.contrib import admin
2 from .models import Student
3
4 admin.site.register(Student)
```

## Create a Superuser

*You need an admin account to log in*

- Run the command from the terminal
- 
- Enter username, email, and password
- 
- Use this account to access /admin
- 
- Keep credentials safe

```
1 python manage.py createsuperuser
```

## Customize Admin with ModelAdmin

*Improve the list screen in admin*

- Create a custom admin class
- 
- Use `list_display` to show important columns
- 
- Use `search_fields` and `list_filter` for productivity
- 
- Admin becomes more usable with a few lines

```
1 from django.contrib import admin
2 from .models import Student
3
4 @admin.register(Student)
5 class StudentAdmin(admin.ModelAdmin):
6     list_display = ('id', 'name', 'level',
7                     'department')
8     search_fields = ('name', 'department')
9     list_filter = ('level', 'department')
```

## Typical Admin Workflow

**1. Login to /admin**

**2. Open Student model**

**3. Click Add**

**4. Fill fields**

**5. Save record**

**6. Edit or delete later**

## Using choices for Controlled Values

*Restrict field values to known options*

- Useful for gender, status, level, semester, etc.
- 
- Improves consistency of stored data
- 
- Often shown as dropdown in forms/admin
- 
- Prevents random text variations

```
1 LEVEL_CHOICES = [  
2     (1, 'Level 1'),  
3     (2, 'Level 2'),  
4     (3, 'Level 3'),  
5     (4, 'Level 4'),  
6 ]  
7  
8 level = models.IntegerField(choices=LEVEL_CHOICES)
```

## Timestamps and Ordering

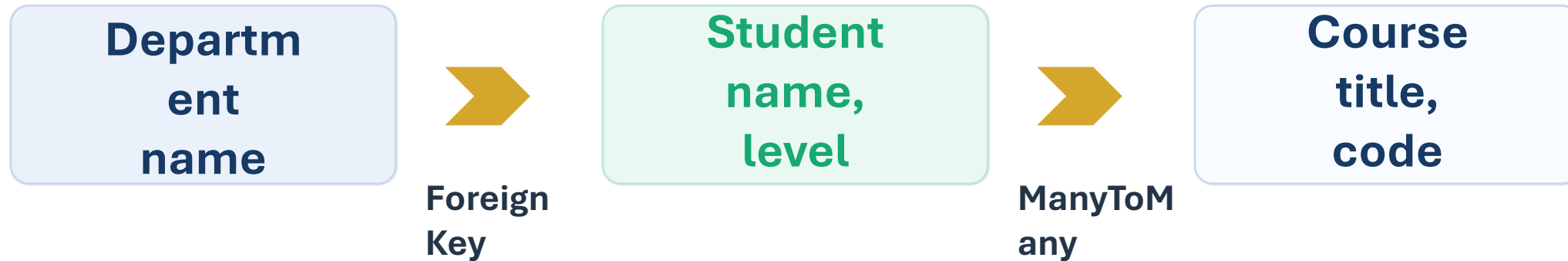
*Useful metadata for real systems*

- created\_at stores when record was created
- 
- updated\_at stores latest modification time
- 
- ordering in Meta controls default sorting
- 
- Very useful in admin and query results

```
1 created_at =  
models.DateTimeField(auto_now_add=True)  
2 updated_at = models.DateTimeField(auto_now=True)  
3  
4 class Meta:  
5     ordering = ['name']
```

## Mini Project Design

*Student + Course + Department*



- Students belong to one department
- 
- Students can register in many courses
- 
- Courses can contain many students

## Practice 1 – Choose the Correct Field Type

**student name**

Students  
answer

**student age**

Students  
answer

**course description**

Students  
answer

**email**

Students  
answer

**created time**

Students  
answer

**Ask learners to answer: CharField? IntegerField? TextField? EmailField? DateTimeField?**

## Practice 2 – Design a Course Model

- Ask students to propose fields for a Course model
- 
- Possible ideas: title, code, credit\_hours, description
- 
- Ask which fields are required and which are optional
- 
- Ask whether any choices or relationships are needed

### Possible Answer

```
1 class Course(models.Model):  
2     title =  
models.CharField(max_length=100)  
3     code =  
models.CharField(max_length=10)  
4     credit_hours = models.IntegerField()  
5     description =  
models.TextField(blank=True)
```

## Quick Quiz 1

**What does `python manage.py makemigrations` do?**

A. Deletes all tables

B. Creates migration files based on model changes

C. Opens the admin panel

D. Creates a superuser

**Expected  
answer: B**

## Quick Quiz 2

**Which relationship fits: one student can take many courses, and one course can have many students?**

A. ForeignKey

B. OneToOneField

C. ManyToManyField

D. CharField

**Expected  
answer: C**

## Lecture Recap

*The six big ideas to leave with*

**1. Models describe data with Python classes**

**3. Migrations sync model changes with the database**

**5. ORM lets us use Python to work with data**

**2. Fields map to database columns**

**4. Relationships connect tables logically**

**6. Admin gives a ready-made management interface**

## Homework

- Create a Department model and a Student model
- Connect Student to Department using ForeignKey
- Register both models in admin.py
- Run makemigrations and migrate
- Create a superuser and add at least 3 departments and 5 students from the admin panel
- Write down which field types you used and why

## Preview of Lecture 9

*Forms and CRUD Operations*

- How users submit data using forms
- 
- How backend receives and validates form data
- 
- How to build create, read, update, and delete pages
- 
- Connecting forms to models and templates

**Today we created the data structure.**

**Next lecture, users will interact with it.**

## Questions and Discussion

**Ask anything about models, migrations, database logic, or admin workflow.**

Suggested prompts:

- What was the most confusing point today?
- Which command do you want to practice again?
- Which relationship still needs clarification?